**G S Dheekshith**
**PES2UG24CS166**

# SDN mini-Project Report:
# Simple QoS Priority Controller using mininet & POX

## Objective:

The aim of this project is to build a simple SDN-based system that can prioritize network traffic. The goal is to show that different types of traffic can be given different bandwidth using a controller and Open vSwitch.

## Introduction:

In normal networks, all traffic is treated the same. This becomes a problem when important applications like video calls or online meetings get slowed down because of background traffic.

Quality of Service (QoS) solves this by giving priority to certain types of traffic.

Software Defined Networking (SDN) makes this easier by separating the control logic from the actual network devices. Instead of configuring each device manually, a controller can decide how traffic should be handled.

## Flow:

```
dheekshith@asus-a15:~/pox$ nano ~/pox/pox/misc/qos_controller.py
dheekshith@asus-a15:~/pox$ cd ~/pox
python3 pox.py log.level --DEBUG openflow.of_01 misc.qos_controller
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.10.12/Mar 3 2026 11:56:32)
DEBUG:core:Platform is Linux-6.8.0-107-generic-x86_64-with-glibc2.35
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.10.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:misc.qos_controller:Switch connected
DEBUG:openflow.of_01:1 connection aborted
```

First we start the controller(POX);

```
dheekshith@asus-a15:~/sdn-qos-pox$ cd ~/sdn-qos-pox
sudo mn -c
sudo mn --custom topo.py --topo qostopo --controller remote,ip=127.0.0.1,port=6633
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
***  Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

On a Second terminal we try starting the mininet; where we create the network topology.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Apply QOS and verify if it worked;

```
mininet> sh ovs-vsctl set port s1-eth2 qos=@newqos -- --id=@newqos create qos type=linux-htb other-config:max-rate=10000000 queues:1=@q1 queues:2=@q2 -- --id=@q1 create queue other-config:min-rate=7000000 -- --i
d=@q2 create queue other-config:max-rate=1000000
dc266a77-bfc0-44cf-be80-5f7b56e47e84
ac926406-8436-41b9-af72-1770648ad123
46c04aa3-0649-4b19-90a3-a00f8c0f5097
mininet> sh ovs-vsctl list queue
_uuid               : 2f5edc6e-8418-48a6-b30d-bbb7e575d130
dscp                : []
external_ids        : {}
other_config        : {min-rate="7000000"}

_uuid               : ac926406-8436-41b9-af72-1770648ad123
dscp                : []
external_ids        : {}
other_config        : {min-rate="7000000"}

_uuid               : 46c04aa3-0649-4b19-90a3-a00f8c0f5097
dscp                : []
external_ids        : {}
other_config        : {max-rate="1000000"}

_uuid               : 8a37a1eb-b537-4956-8f49-af0569895392
dscp                : []
external_ids        : {}
other_config        : {max-rate="1000000"}
```

Start traffic servers (h2)

```
Activities    XTerm

root@asus-a15:/home/dheekshith/sdn-qos-pox# iperf -s -p 5001
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.2 port 5001 connected with 10.0.0.1 port 37432
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-12.2343 sec  14.0 MBytes  9.60 Mbits/sec
```

```
Activities    XTerm

root@asus-a15:/home/dheekshith/sdn-qos-pox# iperf -s -p 5002
------------------------------------------------------------
Server listening on TCP port 5002
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.2 port 5002 connected with 10.0.0.1 port 43738
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-20.7515 sec  2.38 MBytes   960 Kbits/sec
```

```
Activities    XTerm

root@asus-a15:/home/dheekshith/sdn-qos-pox# iperf -c 10.0.0.2 -p 5001
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 47646 connected with 10.0.0.2 port 5001
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-11.7459 sec  13.4 MBytes  9.55 Mbits/sec
root@asus-a15:/home/dheekshith/sdn-qos-pox# iperf -c 10.0.0.2 -p 5002
------------------------------------------------------------
Client connecting to 10.0.0.2, TCP port 5002
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[  1] local 10.0.0.1 port 34326 connected with 10.0.0.2 port 5002
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-20.7525 sec  2.38 MBytes   960 Kbits/sec
root@asus-a15:/home/dheekshith/sdn-qos-pox#
```

The experiment was conducted by first starting the controller, followed by the Mininet topology. After verifying connectivity, QoS queues were configured on the switch. Traffic was then generated using iperf, and throughput differences were observed between high and low priority traffic.

As high priority queues had higher bandwidth ranging upto 9.55Mbps,  whereas 960Kbps in case of the bandwidth of the lower priority queue.

## Code Scripts:

*qos_controller.py
~/sdn-qos-pox

```python
from pox.core import core
import pox.openflow.libopenflow_01 as of

log = core.getLogger()

HIGH_PRIORITY_PORT = 5001
LOW_PRIORITY_PORT = 5002


def install_flow(connection, out_port, tcp_port, priority):
    msg = of.ofp_flow_mod()
    msg.priority = priority
    msg.match.dl_type = 0x800
    msg.match.nw_proto = 6
    msg.match.tp_dst = tcp_port
    msg.actions.append(of.ofp_action_output(port=out_port))
    connection.send(msg)


def _handle_ConnectionUp(event):
    log.info("Switch connected")

    install_flow(event.connection, 2, HIGH_PRIORITY_PORT, 100)

    install_flow(event.connection, 2, LOW_PRIORITY_PORT, 10)


def launch():
    core.openflow.addListenerByName("ConnectionUp", _handle_ConnectionUp)
```

topo.py
~/sdn-qos-pox

```python
from mininet.topo import Topo

class QoSTopo(Topo):
    def build(self):
        s1 = self.addSwitch('s1')
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')

        self.addLink(h1, s1)
        self.addLink(h2, s1)

topos = {'qostopo': QoSTopo}
```

## References:

GITHUB: [QOS-repo link]